

Project 8 (Bonus) — MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically

Title: MCPBridge Agent That Discovers And Uses Tools Automatically

Type: Single agent (ReAct loop over dynamically discovered MCP tools) · **Stack:** Python, `mcp` (the official MCP Python SDK), OpenAI SDK (function calling), `python-dotenv`, `pydantic`, `asyncio` · **Level:** Intermediate–Advanced **Tagline:** A ReAct agent whose tools aren't written into its own Python code — it connects to one or more MCP servers at startup, discovers what they offer, and calls them dynamically.

Not part of the numbered Doc01-19 project arc — a bonus/portfolio project built on [06_tools_function_calling](#)'s MCP sections and [07_ai_agents](#)'s ReAct loop, and the direct companion to Project 7. Read Doc06 and Doc07's Core Concepts first if you haven't already — this file has the full build spec, not a pointer elsewhere.

Charter (what this project is)

Build a real single-agent ReAct loop (the same think → act → observe pattern from Doc07), except its tools aren't a hardcoded Python list — they're discovered at startup from one or more live MCP servers, and called through the MCP protocol instead of a direct function call. Nothing about the *loop* changes from Doc07; what changes is where the tools come from and how a tool call actually gets executed. This is the natural companion to Project 7 (an MCP server other people build, exposing its own tools over the protocol) — this project is the client that actually uses a server like that, or any other real MCP server, including public reference servers.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

The Story — what you're actually building

Every agent you've built so far in this curriculum has its tools written directly into its own code — a `@tool`-decorated Python function, sitting right there in `tools.py`, that the agent's loop calls directly. That's fine when you own the tool. It stops being fine the moment the tool you want lives somewhere else — a teammate's notes server, a company-wide search index, a filesystem service someone else runs — and you don't want to reimplement it, or worse, copy-paste its logic into every agent that needs it.

MCP exists exactly for that handoff. A server stands somewhere (could be a subprocess on your own machine, could be a shared service) and exposes its tools, resources, and prompts over one standard protocol. Any client — including an agent you build — can connect to it, ask "what have you got?", and use whatever comes back, without either side needing to know the other's internals ahead of time. That's the whole idea behind MCPBridge: an agent that never hardcodes a single tool. It connects, it asks, it uses whatever it's handed — the same ReAct loop from Doc07, just pointed at a *discovered* tool list instead of a written one.

This matters because it's the real shape of tool access in a lot of production agent systems: the agent and the tool provider are built and deployed by different people, possibly on different schedules, and the only contract between them is the protocol. The Steps below build this up the same way Project 2 built up the Worker agent: Step 1 proves discovery alone works, with no agent yet. Step 2 wires that discovery into a real ReAct loop that picks and calls a tool it never knew about at write-time. Step 3 faces the practical problem of needing tools from *more than one* server at once. Step 4 makes the whole thing survive a server that isn't there, or that drops mid-task — and ties it all together into one working demo.

Before you read further — think about it yourself: if your agent's tool list isn't known until the program actually runs and connects to a server, how does that change the code you write to hand tools to the model — compared to every earlier project, where the tool list was just written directly in `tools.py` ? And if two different MCP servers both happen to expose a tool called `search` , how should your agent decide which one a given tool call is actually supposed to hit? Sit with both of these before you read the Steps below.

What you're actually building, in one line: a ReAct agent whose tool list comes from calling live MCP servers at startup, not from functions written into its own code.

Why this needs to exist: real tools often live somewhere the agent's own codebase can't reach — a teammate's server, a shared company service — and hardcoding a copy of each one means every change the tool's owner makes has to be copied over by hand, everywhere it's used.

When you'd reach for this at a real job: when an agent needs a capability owned by a different team or service, and you want it to pick up new or changed tools automatically instead of shipping a code change every time the tool provider updates something.

How it works, mechanically: at startup the agent connects to each configured MCP server, calls `list_tools()` to get each tool's name, description, and schema, converts that schema into the shape the model's function-calling expects, and routes any tool call the model makes back to the right server.

Why not just do it some simpler/different way: the obvious alternative is hardcoding the tool list you already know about — if you already know what tools exist, why discover them at runtime at all? That works fine right up until the server adds, removes, or changes a tool: a hardcoded list quietly

goes stale, and nobody notices until a call fails. Discovering tools at runtime means the agent always matches what the server actually offers right now — which matters most exactly when the agent and the tool provider are built and shipped by different people on different schedules.

Where This Fits

This isn't part of the main 5-project arc (Projects 1-5), and it isn't scored the way Project 6 tests whether the whole multi-agent architecture generalizes to a new problem. This project is narrower and more specific on purpose: it's a deep, hands-on look at **one real integration technology** — MCP — that's increasingly how production agents actually get their tools, rather than every tool being hand-coded per app. It belongs in your portfolio as proof you can build the *consuming* side of a protocol, not just call functions you wrote yourself.

It's designed as the direct companion to **Project 7** (an MCP server other people can build — exposing its own tools, resources, and prompts over the protocol). Project 7 answers "how do I expose my own tools to any MCP client?" This project answers the other half: "how do I build a client agent that actually uses one?" You can point MCPBridge straight at Project 7's server once it exists, or at any other real MCP server — a simple one you write yourself in a few lines with `MCPServer` (Doc06 shows the minimal shape), or a public reference server like the official filesystem or fetch servers. Nothing here depends on Project 7 existing first; MCP is a real, standard protocol, and any conforming server works.

Setup (do this once, before Step 1)

```
cd project_8_mcpbridge
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install mcp openai python-dotenv pydantic
pip freeze > requirements.txt
```

Same `.env` setup as every other project (`OPENAI_API_KEY` , `LOG_LEVEL`). You'll also need at least one real MCP server to connect to before Step 1. Pick one of these: - **Project 7's server**, if you've built it (or a teammate has) — point at it directly. - **A public reference server** — e.g. the official filesystem server, run with `npx -y @modelcontextprotocol/server-filesystem /some/local/dir` (needs Node.js installed). - **A tiny server you write yourself** — the minimal `MCPServer` example from [06_tools_function_calling's Core Concepts](#) is a complete, working server in under 15 lines. Save it as `example_server.py` in this folder and use it for Steps 1-2.

This project is `asyncio`-based throughout, because the `mcp` SDK's client is async-only. If `async` / `await` still feels unfamiliar, [08b_async_prereq's Core Concepts](#) is a quick, optional read — but Doc06's own MCP client example already shows the exact pattern (`asyncio.run(main())`)

wrapping an `async def main()` you'll be reusing throughout this project, so it's not a hard prerequisite.

A Real Example (so this isn't just theory)

Use this scenario, or one close to it, for every step below — keep it this specific so you have real test cases, not made-up ones:

Scenario: you're connecting MCPBridge to two small MCP servers: a **notes** server (tools like `search_notes(query: str)` and `read_note(note_id: str)`, backed by a folder of a few `.md` files you write yourself) and a **filesystem** server (tools like `list_files(directory: str)` and `read_file(path: str)`, the public reference server or your own small version of it).

Test these tasks (save them in `test_tasks.py` so you reuse the same ones every run): 1. "What does my note about the Q3 roadmap say?" — needs exactly one server (notes): `search_notes` to find it, then `read_note` to read it. 2. "List the files in the project folder, then tell me if any of them look like a config file." — needs exactly one server (filesystem): `list_files`, then reasoning over the names, no `read_file` call needed. 3. "Check my notes for anything about the deploy process, and also check if there's a deploy script in the project folder." — needs **both** servers in one run — this is the case that actually proves namespacing (Step 3) and cross-server routing work, not just that each server works alone. 4. "Look something up in my notes" — asked while the notes server is deliberately not running — this is Step 4's test: the agent should say clearly it can't reach the notes tools, not hang or crash, and should still be able to use the filesystem tools if the task needs them too.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** shows you can build the *consumer* side of a real, standardized tool protocol — not just call functions you wrote yourself, which is what every earlier project in this curriculum does.
- **Why it matters:** MCP is a genuinely new, fast-spreading standard (Claude Desktop, IDEs, and a growing list of vendor tools all speak it) — an interviewer who's heard of it will immediately understand what this project proves, and one who hasn't will still recognize "an agent that discovers its own tools at runtime instead of hardcoding them" as a real, non-toy design.
- **When you'd build something like this at a real job:** any agent that needs to reach tools owned by a different team, a different service, or a third party — internal platform teams increasingly expose shared capabilities as MCP servers precisely so consuming teams don't have to hand-wire an integration each time.
- **How it's built:** a stdio connection to each server, a discovery call that turns whatever tools exist into the agent's tool list at runtime, a hand-built ReAct loop (reusing Doc07's pattern) that calls

tools through the MCP client instead of a direct Python call, and a fallback layer so one unreachable server degrades the agent instead of crashing it.

Problems you'll likely run into, and how to fix them:

Problem	Fix
A discovered tool's <code>inputSchema</code> doesn't look like what OpenAI's <code>tools</code> parameter expects, and the API call fails	Both are already JSON Schema — write one small converter function (<code>mcp_tools_to_openai_schema</code>), test it against the <i>real</i> schemas your server returns, and don't assume the shapes match without checking
Two servers both expose a tool called <code>search</code> , and the model calls one when you meant the other	Namespace every discovered tool name before it reaches the model (Step 3), and keep a routing table mapping the namespaced name back to the exact server + original tool name
The MCP server process never finishes starting up (a bad command, a crash on launch), and your whole agent hangs forever	Wrap the connection and <code>session.initialize()</code> in <code>asyncio.wait_for(...)</code> with a real timeout — a timeout here means "this server is unavailable," not a bug to retry forever
A tool call returns successfully, but your code crashes trying to print it as a plain string	<code>call_tool()</code> returns a result object with a <code>.content</code> list of content blocks, not a plain string — extract the text explicitly instead of assuming
The agent keeps trying to call a tool from a server that dropped mid-task	Catch that failure at the call site, feed a clear error back to the model as the tool's observation (same instinct as Doc06's "getting a tool's failure back to the model, correctly"), and drop that server's tools from what's offered for the rest of the run

Built During These Documents

[06_tools_function_calling](#) → [07_ai_agents](#)

Plan Before You Code

Same process as every project (see [15_five_projects_index](#)): Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks — before writing the connection code.

How To Build This — Step by Step

Step 1 — Connect to One MCP Server and List Its Tools

Project: MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically — Step 1 of 4: Connect to One MCP Server and List Its Tools

What this step does: proves the smallest possible piece works before anything else gets built on top of it — a real client connecting to a real server over stdio, and discovering what it offers. No agent loop yet, no model call at all — just the connection and the discovery. **Why this step matters:** every later step in this project depends on this connection working correctly. If discovery is broken, debugging Step 2's agent loop on top of it means debugging two things at once instead of one — the same reason Project 2's Step 1 proved its chain alone before any loop touched it. **When you'll hit this for real:** the first few minutes of connecting to *any* new MCP server, in any project — before you trust it enough to build an agent on top of it, you confirm the connection itself actually works and returns what you expect. **Read first:** [06_tools_function_calling Core Concepts — "MCP servers and clients, and how they actually connect"](#), [06_tools_function_calling Core Concepts — "What MCP actually is, and the problem it solves"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Pick your one MCP server for now (Setup's `example_server.py`, the filesystem reference server, or Project 7's, if it exists). Write a small helper that launches it as a subprocess over stdio using `StdioServerParameters` + `stdio_client`, opens a `ClientSession` on top, and calls `await session.initialize()` — the required handshake before anything else on the session works. 2. Call `await session.list_tools()` and print each tool's `name`, `description`, and `inputSchema` — this is the exact information Step 2 will hand to the model, so look closely at its real shape now, not just what you assume it looks like. 3. Run it twice: once against a server that starts correctly, and once against a deliberately wrong command (a typo'd script path) — confirm you get a clear Python exception, not a silent hang, so you know what "this server isn't working" actually looks like before Step 4 has to handle it gracefully.

Your files after Step 1:

```
project_8_mcpbridge_mcp_client/
├─ example_server.py      → (only if you're not using Project 7's or a public s
├─ mcp_connection.py      → connect_stdio_server(command, args) -> ClientSessi
└─ discover.py            → connects to one server, calls list_tools(), prin
```

Step 2 — A Real ReAct Agent That Calls a Discovered Tool

Project: MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically — Step 2 of 4: A Real ReAct Agent That Calls a Discovered Tool

What this step does: wires Step 1's discovered tools into an actual think → act → observe loop (Doc07's pattern) — the model picks a tool from what was discovered at runtime, not from anything

written in this project's own code, and the agent executes that choice through the MCP client. **Why this step matters:** this is the actual point of the whole project — proving an agent can act on tools it never had written into it. Step 1 only proved *you* can see what a server offers; this step proves the *model* can use it, live, without you writing a single `@tool`-decorated function for it. **What's new vs. Step 1:** a model call, a loop, and a bridge that turns MCP's tool shape into the shape OpenAI's function-calling expects. **What stays the same:** Step 1's connection code — you're building on top of a working connection, not replacing it. **When you'll hit this for real:** any agent that's meant to plug into a tool source it doesn't control — the moment "call this specific Python function" becomes "call whatever this server happens to expose." **Read first:** [07_ai_agents Core Concepts — "The ReAct loop: think → act → observe"](#), [07_ai_agents Core Concepts — "The scratchpad: how the loop remembers its own steps"](#), [07_ai_agents Core Concepts — "Setting a limit on the number of steps"](#), [06_tools_function_calling Core Concepts — "MCP's 3 building blocks: Tools, Resources, and Prompts"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `mcp_tools_to_openai_schema(tools)` : for each MCP tool, build the `{"type": "function", "function": {"name": ..., "description": ..., "parameters": ...}}` dict OpenAI's `tools` parameter expects — an MCP tool's `inputSchema` is already JSON Schema, so this is mostly a rename, not a rewrite. 2. Write `call_mcp_tool(session, name, arguments)` : calls `await session.call_tool(name, arguments)` and returns the result as a plain string — remember the result comes back as a `CallToolResult` with a `.content` list of content blocks (usually `TextContent`, with a `.text` attribute), not a plain string, so extract that explicitly. 3. Build the loop yourself (same shape as Project 2's Step 2 — no `create_agent / AgentExecutor`) : send the conversation + the discovered tool schemas to the model, check for tool calls in the response, run any tool call through `call_mcp_tool`, append the result to the message history as a `tool` role message, and repeat until the model answers with no more tool calls. Add a simple `max_iterations` limit around the loop now (Doc07's "a limit isn't optional") — a plain, hard-coded cap is enough here; Step 4 is where it gets hardened into a named error with the partial log attached. 4. Test it against a task from **A Real Example** above that needs exactly one tool call, and one that needs two chained calls (like `search_notes` then `read_note`) — confirm the second call's arguments actually use information from the first call's result, proving the scratchpad is really being fed back in.

Your files after Step 2:

```
project_8_mcpbridge_mcp_client/
├─ example_server.py
├─ mcp_connection.py
├─ tool_bridge.py           → mcp_tools_to_openai_schema(tools) -> list[dict];
├─ agent.py                 → run_agent(task, session, tools, max_iterations)
└─ main.py                  → connects to one server, discovers tools, runs t
```


Step 3 — Multiple MCP Servers at Once, With Tool Namespacing

Project: MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically — Step 3 of 4: Multiple MCP Servers at Once, With Tool Namespacing

What this step does: connects the agent to more than one server at the same time, and solves the real problem that creates — two servers can each expose a tool with the same name, so the agent needs a way to know which server a given tool call actually belongs to. **Why this step matters:** a real agent rarely needs tools from only one source — a notes server, a filesystem server, a search server, each built and run separately. Without namespacing, two same-named tools silently collide, and the agent (or your routing code) can't tell them apart. **What's new vs. Step 2:** a second server connection, and a namespacing/routing layer sitting between the model and `call_mcp_tool`. **What stays the same:** the ReAct loop itself from Step 2 — it still just sees a flat list of tool names and calls one; it doesn't need to know namespacing exists underneath. **When you'll hit this for real:** the moment any agent needs capabilities from more than one team or service at once — which is most real deployments, once an agent grows past a single demo integration. **Read first:** [06_tools_function_calling Core Concepts — "Why MCP matters for the multi-agent systems this curriculum builds"](#), [06_tools_function_calling Core Concepts — "Choosing between multiple tools"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Move your server details into `servers_config.py` — a list like `[{"name": "notes", "command": "python", "args": ["notes_server.py"]}, {"name": "filesystem", "command": "npx", "args": [...]}]` — instead of one hardcoded connection. 2. Build `MCPServerPool` in `server_pool.py`: connects to every server in the config, keeps each `ClientSession` open for the life of the run, and builds one combined tool list where every tool name is namespaced as `f"{server_name}__{tool_name}"`. **Watch this exact gotcha:** OpenAI's function-name field only allows letters, digits, underscores, and hyphens — a `.` (like the human-readable `notes.search_notes` you might picture) isn't valid there, so use `__` as the actual separator in the name the model sees, and keep a routing dict (`{"notes__search_notes": ("notes", "search_notes")}`) mapping back to the real server + tool name for when a call needs dispatching. 3. Update `agent.py` to call through `MCPServerPool.call_tool(namespaced_name, arguments)` instead of `call_mcp_tool` directly — the pool looks up the right session from the routing dict and dispatches there. 4. Test the "needs both servers" task from **A Real Example** (task 3) — confirm from your logs that each tool call actually routed to the server you expected, not just that the final answer happened to look right.

Your files after Step 3:

```
project_8_mcpbridge_mcp_client/
├─ example_server.py
├─ mcp_connection.py
├─ tool_bridge.py
├─ servers_config.py      → SERVERS = [{"name": ..., "command": ..., "args":
├─ server_pool.py         → MCPServerPool: connects to every configured ser
```


<pre> — agent.py — main.py</pre>	<pre>→ calls through MCPServerPool instead of one ses → connects to 2+ servers, runs a task needing t</pre>
-----------------------------------	---

Step 4 — A Fallback When a Server Is Unreachable, and a Full Working Demo

Project: MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically — Step 4 of 4: A Fallback When a Server Is Unreachable, and a Full Working Demo

What this step does: makes the whole system survive the real-world case where a server isn't running, never finishes starting, or drops mid-task — instead of hanging forever or crashing the entire agent — and ties every earlier step into one final, working demo. **Why this step matters:** every step so far assumed every server works. Production doesn't work that way — a server can be down for a deploy, misconfigured, or just slow. An agent that can't tell "one tool source is temporarily gone" from "everything is broken" isn't something you'd trust with a real task. **What's new vs. Step 3:** a timeout + try/except around each server's connection attempt in `MCPServerPool` (a failed server gets logged and skipped, not fatal), and error handling around individual tool calls so a mid-task disconnect becomes a clear observation fed back to the model, not an unhandled exception. **What stays the same:** every server that *does* connect works exactly as it did in Step 3 — resilience wraps around the existing pool and loop, it doesn't rewrite them. **When you'll hit this for real:** literally the first time you demo this project to someone else and one of your servers isn't running yet — which will happen, and is exactly why this step exists before the final demo, not after. **Read first:** [06_tools_function_calling Core Concepts — "Getting a tool's failure back to the model, correctly"](#), [07_ai_agents Core Concepts — "Common single-agent failures, worth naming now"](#), [07_ai_agents Core Concepts — "Setting a limit on the number of steps"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. In `MCPServerPool`, wrap each server's `connect + initialize()` in `asyncio.wait_for(..., timeout=...)` inside a `try / except`. A server that times out or raises gets logged clearly (`"[notes] unreachable: <reason>, continuing without it"`) and is left out of the combined tool list — it does **not** stop the other servers from connecting, and it does **not** crash the pool's setup. 2. In the routing/call path, wrap the actual `call_tool()` dispatch in its own `try / except` too — a server that connected fine at startup can still drop mid-task. On failure, return a clear, labeled error string as the tool's observation (reusing Doc06's "getting a tool's failure back to the model, correctly" instinct) instead of letting the exception crash the whole agent loop. Also keep Doc07's `max_iterations` hard limit from Project 2's Step 3 in place here — a confusing failure is exactly the kind of thing that can make a model retry the same broken call repeatedly without it. 3. Write `main.py` as the final demo: connect to your configured servers (deliberately leave one misconfigured or not started), run task 3 and task 4 from **A Real Example**, and print a clear run summary — which servers connected, which tools were available, every step the agent took, and the final answer. 4. Test it for real: run the full demo once with every server up (confirm it all works), then again with one server killed or never started (confirm the agent finishes, using only what's left, and clearly states it couldn't reach the missing one rather than silently pretending everything worked).

Your files after Step 4 (final):

```

project_8_mcpbridge_mcp_client/
├─ example_server.py
├─ mcp_connection.py
├─ tool_bridge.py
├─ servers_config.py
├─ server_pool.py           → connect wrapped in a timeout + try/except per s
├─ agent.py                 → tool-call failures (including a mid-task drop)
├─ main.py                  → full demo: connects to whatever servers are a
└─ test_agent.py            → kills/omits one server on purpose, confirms

```

Final Deliverable: MCPBridge-Agent-That-Discovers-And-Uses-Tools-Automatically — a single ReAct agent that connects to one or more real MCP servers, discovers their tools at runtime instead of hardcoding any of them, routes namespaced tool calls to the right server, and keeps working on whatever tools remain available when a server is unreachable.

Checklist Before You Call This Done

- [] The agent's tool list comes entirely from `list_tools()` calls at runtime — no tool is written directly into this project's own Python code
- [] The ReAct loop (Step 2) reuses Doc07's think → act → observe shape, hand-built, not `create_agent / AgentExecutor`
- [] At least 2 MCP servers connected at once, with tool names namespaced and correctly routed back to the right server
- [] A test where the agent needs tools from both servers in a single run, and your logs show each call hit the server you expected
- [] A test where one configured server is unreachable — the agent logs it clearly, keeps running on the remaining tools, and doesn't hang or crash
- [] A test where a tool call fails mid-task — the failure is fed back to the model as a clear observation, not an unhandled exception
- [] You can explain, for a namespaced tool call, exactly which piece of code decided which server it should go to

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**